

Large-Scale Web Crawler & TF-IDF Search Engine

UCI ICS 도메인을 대상으로 대규모 웹 페이지를 수집하고, 수집된 문서를 기반으로 TF-IDF 기반 검색 엔진을 구현하는 프로젝트. 단순 크롤링을 넘어 URL 정규화, 중복 제거, Trap URL 탐지, 역색인 구축, 검색 결과 반환까지 검색 엔진의 핵심 파이프라인을 직접 설계 및 구현

핵심 목표

대규모 웹 페이지를 안정적으로 수집하면서 중복 URL, 무한 반복 URL, 비정상 종료, 대용량 색인 처리 문제를 해결하고, 검색 가능한 Inverted Index 구조 구축

- 동일한 페이지가 여러 URL 형태로 중복 수집되는 문제
→ URL 정규화 및 유효성 검사를 통해 중복 URL 제거
- stayconnected/stayconnected/..., datasets/datasets/...처럼 무한 반복되는 Trap URL 발생
→ 반복 path, 과도한 query parameter, cyclic URL pattern 기반 Trap URL 필터링
- 장시간 크롤링 도중 비정상 종료 시 진행 상태 손실 가능성
→ Frontier 상태 저장/복구 기능으로 비정상 종료 이후에도 크롤링 재개 가능하도록 설계
- 수만 개 문서와 수십만 개 단어를 효율적으로 저장하고 검색해야 하는 문제
→ JSON 기반 Inverted Index 구조 설계
- 검색 결과의 관련도를 계산하기 위한 TF-IDF 기반 ranking 필요
→ TF, tag importance, TF-IDF score를 함께 저장하여 검색 결과 ranking에 활용

담당 역할

Crawler Architecture 설계

- frontier.py, crawler.py, corpus.py, main.py로 역할을 분리하여 크롤링 파이프라인 모듈화

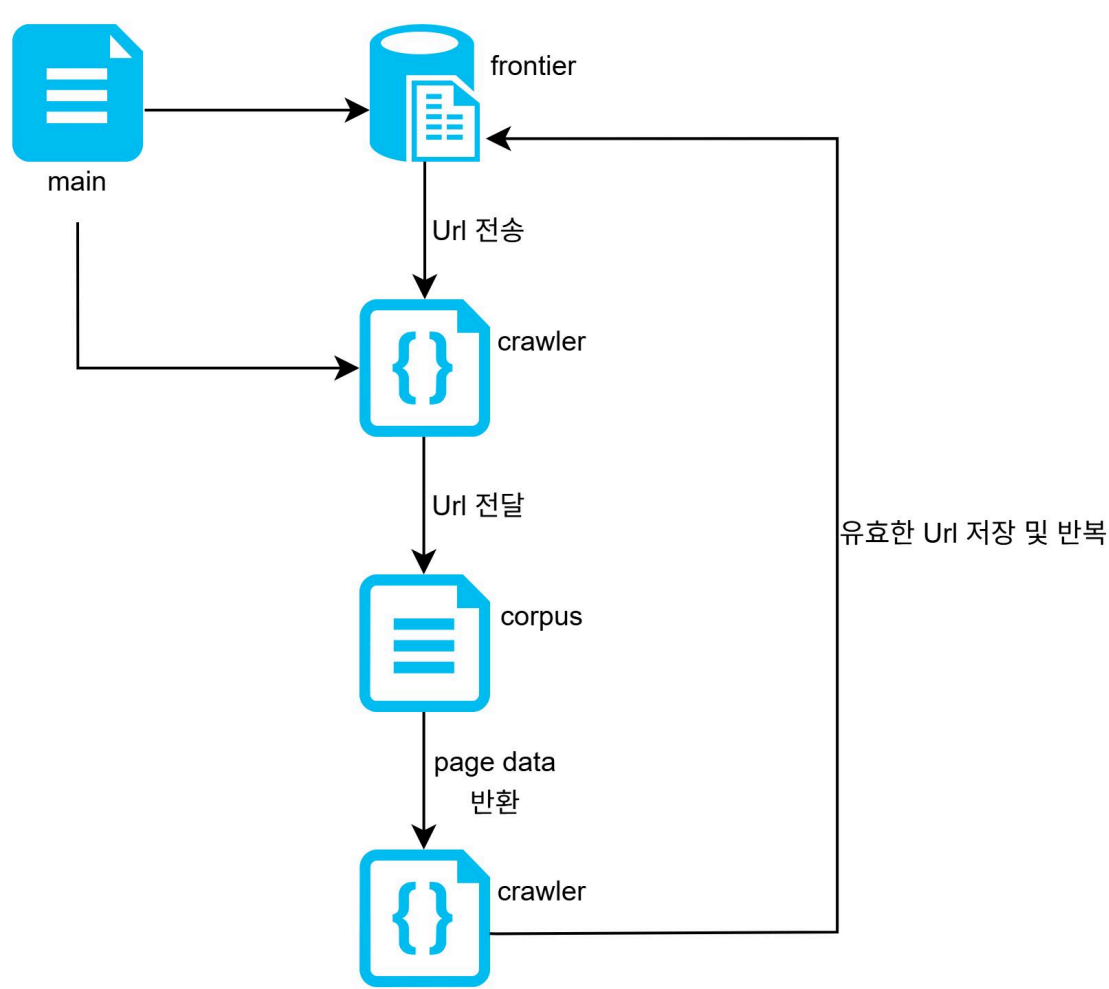
검색 엔진 핵심 알고리즘 구현

- TF-IDF 기반 검색 점수 계산, HTML tag importance를 반영한 ranking 보정 및 Inverted Index 구조 설계/ JSON 파일 저장
핵심 알고리즘 구현

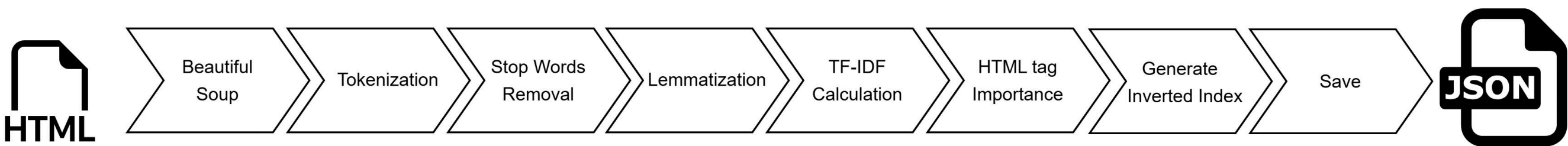
프로젝트 설계

크롤링 엔진

- 1. main.py가 프로그램을 시작 → frontier.py와 crawler.py를 생성
- 2. frontier.py는 앞으로 방문할 URL 목록을 관리하며, crawler.py는 frontier로부터 URL을 받아 corpus.py에 페이지 데이터를 요청
- 3. corpus.py는 HTML 데이터를 반환하고, crawler는 이를 분석하여 새로운 링크를 추출하고 유효성을 검사
- 4. 유효한 URL만 다시 frontier에 저장하며 이 과정을 반복



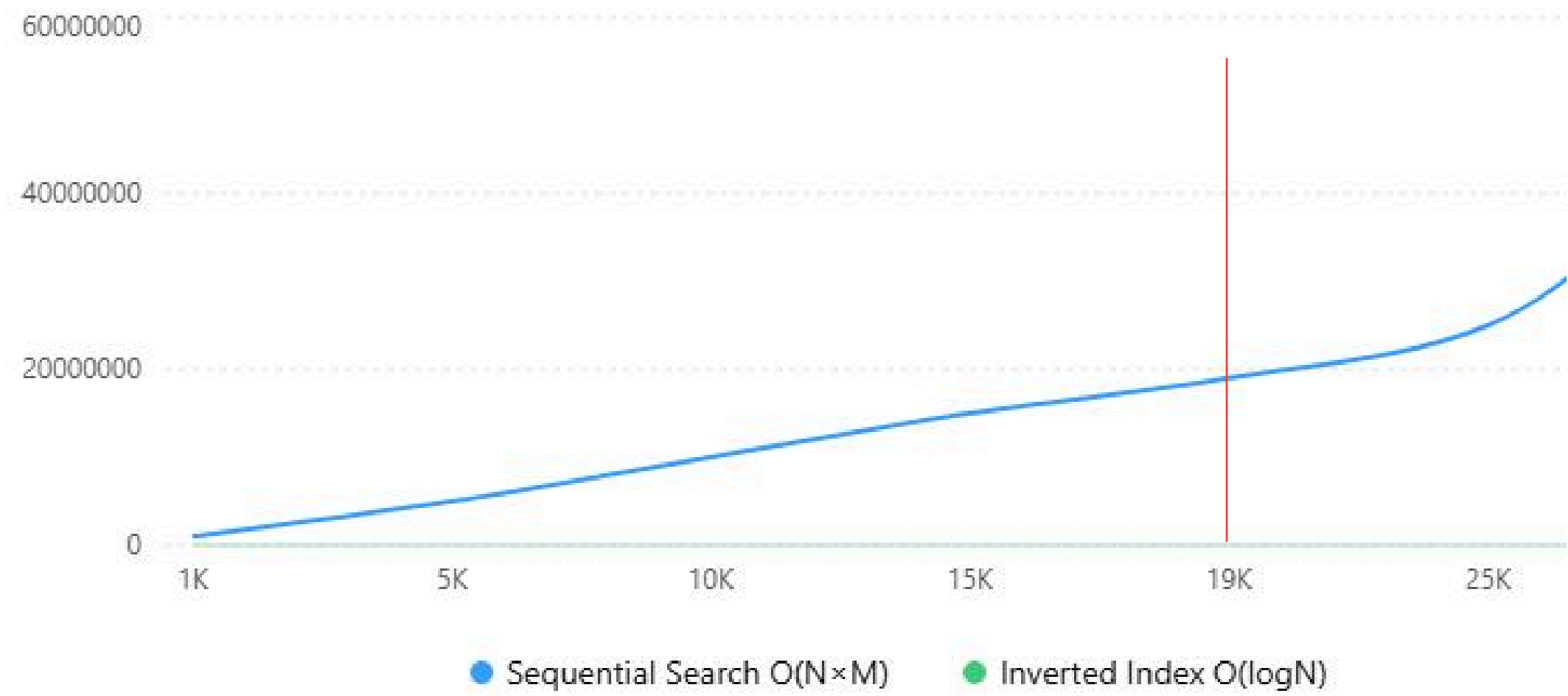
검색 엔진



- 1. **Beautiful Soup**
HTML 문서 분석/ 태그 파싱
- 2. **Tokenization**
본문 텍스트를 단어 단위로 분리 → 분석 가능한 개별 단어(token) 형태로 변환
- 3. **Stop Words Removal**
검색에 큰 의미가 없는 일반 단어(예: the, is, and)를 제거 → 검색 정확도와 인덱싱 효율 상승
- 4. **Lemmatization**
단어를 원형(base form)으로 변환(예: running → run, cars → car) → 검색 품질을 향상
- 5. **TF-IDF Calculation**
단어의 중요도 계산
- 6. **HTML Tag Importance**
HTML 태그별 중요도 반영 (title, h1, bold 태그 안에 포함된 단어는 일반 본문보다 더 중요한 단어로 판단하여 가중치 부여)
- 7. **Generate Inverted Index**
검색엔진용 inverted index 생성
- 8. **Save (JSON)**
최종 inverted index 데이터를 JSON 파일로 저장

핵심 수치

- 1. 약 120,000개 후보 URL 처리
- 2. 중복 URL 78% 이상 제거
- 3. 약 6,200개 Trap URL 필터링
- 4. 약 19,000개 페이지 크롤링



Inverted Index를 이용한 시간복잡도 감소

$$O(N \times M) \rightarrow O(\log N) \text{ (hash collision을 고려한 시간복잡도)}$$

문서 수 = N, 문서당 단어 수 = M

결과

- 총 35,434개 문서 색인 구축
- 총 데이터 크기 약 778147.5 KB 처리
- 389,493개 unique word 추출 및 색인화
- "informatics" 검색어에 대해 6,096개 URL 반환
- "Irvine" 검색어에 대해 12,270개 URL 반환
- "artificial intelligence" 검색어에 대해 699개 URL 반환
- "computer science" 검색어에 대해 14,680개 URL 반환

한계 및 회고

1. Ranking 고도화의 한계

- 기본 TF-IDF와 tag importance를 활용했지만, 사용자의 검색 의도나 문서 품질을 충분히 반영하지 못함
- PageRank, BM25 같은 고도화된 ranking 알고리즘은 적용하지 못함

2. Trap URL 탐지 기준의 일반화 한계

- 반복 path나 query parameter 기반 필터링은 효과적이었지만, 모든 사이트 구조에 일반화되기는 어려움
- 특정 도메인에서는 정상 URL도 과도하게 필터링될 가능성이 있음.

3. 대규모 색인 저장 방식의 한계

- JSON 기반 저장 방식은 구현과 디버깅이 쉬웠지만, 데이터가 커질수록 검색 속도와 메모리 효율에 한계가 있음
- 실제 서비스 수준에서는 DB, Elasticsearch, Lucene 같은 검색 전용 저장소가 더 적합함

개선 방향

1. Ranking 고도화의 한계

- TF-IDF 외에 BM25, PageRank, anchor text 기반 ranking을 추가해 검색 품질 향상
- 검색어와 문서 제목, heading tag, 본문 위치를 함께 고려하는 scoring 방식 개선

2. Trap URL 탐지 기준의 일반화 한계

- 반복 패턴 탐지 기준을 rule-based에서 statistical 방식으로 확장
- 도메인별 URL 구조를 분석해 adaptive filtering 적용

3. 대규모 색인 저장 방식의 한계

- JSON 기반 Inverted Index를 검색 전용 엔진 또는 key-value store로 이전
- 색인 생성과 검색 서버를 분리하여 대규모 데이터 처리 구조로 확장

배운점

1. 크롤링과 검색엔진 구조를 직접 구현하며 URL 관리, Trap 탐지, 상태 복구 등 실제 웹 데이터 처리 과정의 중요성 이해
2. TF-IDF와 Inverted Index를 적용해 대규모 문서 환경에서 검색 성능을 최적화하는 데이터 처리 및 검색 구조 설계 경험